Building a RAG Assistant with Snowflake Cortex Search

1. Introduction

Large Language Models (LLMs) are great at conversation, but they struggle with specific, private business data and can make up facts, which is known as hallucination. This guide shows how to build a Retrieval-Augmented Generation (RAG) assistant using Snowflake Cortex Search and Streamlit.

By grounding the LLM in your own files, the assistant gives accurate, reliable answers, and your data never leaves the Snowflake security perimeter.

The finished product is a chat app called **Chatbot** that:

- Answers questions from your PDF manuals and guides.
- Remembers the conversation, so follow-ups like "what about the cheaper one?" work.
- Lets users filter by product category and choose the LLM.
- Has a clean, modern interface with a welcome screen and suggested questions.

2. How It Works

A RAG application has two phases: preparing the documents once, then using them every time someone asks a question.

Preparation (one time, or whenever documents change)

1. PDFs are uploaded to a Snowflake stage.
2. Each PDF is parsed into text, split into small chunks and labeled with a category.
3. The chunks are stored in a table, and a Cortex Search service indexes them.

Answering a question (every time)

1. The user types a question in the Streamlit app.
2. If memory is on and the chat already has messages, the LLM rewrites the question into a standalone search query using the chat history.
3. Cortex Search returns the most relevant chunks, optionally filtered by category.
4. The chunks, the recent chat history and the question are combined into a prompt.
5. The LLM answers using only that context.

Snowflake creates and manages the embeddings and vector index automatically. There are no external vector databases or embedding pipelines to maintain.

3. Prerequisites

- A Snowflake account in a region that supports Cortex LLM functions.
- A role with access to the Snowflake Cortex AI functions (the SNOWFLAKE.CORTEX_USER database role).
- A warehouse to run queries, the search service refresh and the Streamlit app. A Small warehouse is enough.

- PDF files to use, such as product manuals or technical documents.

4. Organizing Documents

Before asking questions, create a place for the documents in Snowflake.

Step 1. Gather your PDF files. Collect the PDFs you want the assistant to learn from. The Snowflake quickstart provides sample bike and ski manuals, or you can use your own.

Step 2. Create the database and schema.

```
CREATE DATABASE CC_QUICKSTART_CORTEX_SEARCH_DOCS;
CREATE SCHEMA DATA;
```

Step 3. Create a stage with a directory table. The directory table lets Snowflake track the files that arrive.

```
create or replace stage docs
  ENCRYPTION = (TYPE = 'SNOWFLAKE_SSE')
  DIRECTORY = ( ENABLE = true );
```

Step 4. Upload the PDFs. In Snowsight, go to Catalog, then Database Explorer. Open the CC_QUICKSTART_CORTEX_SEARCH_DOCS database, the DATA schema, then Stages, then DOCS. Click **+ Files** and add your PDFs.

Step 5. Verify the upload.

```
ls @docs;
```

You should see every PDF you uploaded in the list.

5. Pre-processing Documents

Raw PDFs must be parsed and split into chunks so the search engine and the LLM can use them.

5.1 Extract the content

The parse function reads each PDF straight from the stage. LAYOUT mode extracts readable text and keeps the document structure.

```
CREATE OR REPLACE TEMPORARY TABLE RAW_TEXT AS
WITH FILE_TABLE as (
  (SELECT
        RELATIVE_PATH,
        SIZE,
        FILE_URL,
        build_scoped_file_url(@docs, relative_path) as scoped_file_url,
        TO_FILE('@DOCS', RELATIVE_PATH) AS docs
    FROM DIRECTORY(@DOCS))
)
SELECT
    RELATIVE_PATH,
    SIZE,
    FILE_URL,
    scoped_file_url,
    TO_VARCHAR (
        SNOWFLAKE.CORTEX.AI_PARSE_DOCUMENT (
```

```
            docs,
            {'mode': 'LAYOUT'} ):content
        ) AS EXTRACTED_LAYOUT
FROM FILE_TABLE;
```

This guide uses AI_PARSE_DOCUMENT, the current name of the older PARSE_DOCUMENT function. Newer Snowflake accounts block the legacy function names.

5.2 Create the chunks table

```
create or replace TABLE DOCS_CHUNKS_TABLE (
    RELATIVE_PATH VARCHAR(16777216), -- path to the PDF file
    SIZE NUMBER(38,0),               -- size of the PDF
    FILE_URL VARCHAR(16777216),      -- URL for the PDF
    SCOPED_FILE_URL VARCHAR(16777216),
    CHUNK VARCHAR(16777216),         -- piece of text
    CHUNK_INDEX INTEGER,             -- position of the chunk
    CATEGORY VARCHAR(16777216)       -- used for filtering
);
```

5.3 Split the text into chunks

Long text is not an effective context for an LLM. SPLIT_TEXT_RECURSIVE_CHARACTER breaks it into smaller, overlapping pieces that are inserted into the table.

```
insert into docs_chunks_table (relative_path, size, file_url,
                               scoped_file_url, chunk, chunk_index)
select relative_path,
       size,
       file_url,
       scoped_file_url,
       c.value::TEXT as chunk,
       c.INDEX::INTEGER as chunk_index
from
    raw_text,
    LATERAL FLATTEN( input => SNOWFLAKE.CORTEX.SPLIT_TEXT_RECURSIVE_CHARACTER (
          EXTRACTED_LAYOUT,
          'markdown',
          1512,
          256,
          ['\n\n', '\n', ' ', '']
       )) c;
```

The numbers 1512 and 256 are the chunk size and the overlap between chunks. Snowflake recommends chunks of no more than about 512 tokens (roughly 385 English words) for best semantic search, so try smaller values if answers are not precise enough.

5.4 Label each document with a category

An LLM labels each document, for example as Bike or Snow. The labels let users filter searches later. Classification is optional for Cortex Search, but it enables filtered, hybrid search.

```
CREATE OR REPLACE TEMPORARY TABLE docs_categories AS
WITH unique_documents AS (
  SELECT DISTINCT relative_path, chunk
  FROM docs_chunks_table
```

```
  WHERE chunk_index = 0
),
docs_category_cte AS (
  SELECT
    relative_path,
    TRIM(snowflake.cortex.AI_CLASSIFY (
      'Title:' || relative_path || 'Content:' || chunk, ['Bike', 'Snow']
     )['labels'][0], '"') AS CATEGORY
  FROM unique_documents
)
SELECT * FROM docs_category_cte;
```

Check that the labels look right, then save them to the chunks table:

```
select * from docs_categories;

update docs_chunks_table
  SET category = docs_categories.category
  from docs_categories
  where docs_chunks_table.relative_path = docs_categories.relative_path;
```

Replace 'Bike' and 'Snow' with labels that fit your own documents.

6. Creating the Cortex Search Service

Setting up search needs no external vector database.

```
create or replace CORTEX SEARCH SERVICE CC_SEARCH_SERVICE_CS
ON chunk
ATTRIBUTES category
warehouse = COMPUTE_WH
TARGET_LAG = '1 minute'
as (
    select chunk,
           chunk_index,
           relative_path,
           file_url,
           category
    from docs_chunks_table
);
```

Part	Meaning
CC_SEARCH_SERVICE_CS	The name of the service the app connects to.
ON chunk	The text column that is indexed and searched.
ATTRIBUTES category	Makes category available as a filter.
warehouse	Used to keep the index up to date. Replace COMPUTE_WH with your own warehouse.
TARGET_LAG	How fresh the index should be: it refreshes about every minute.

Snowflake generates the embeddings and builds the index automatically. The service combines semantic (meaning-based) and keyword search for high-quality retrieval.

7. Building the Chat UI

The app runs in Streamlit in Snowflake, so it stays inside your account and respects your role-based access controls.

7.1 Create the Streamlit app

1. In Snowsight, open Catalog, then Streamlit, and click **+ Streamlit App**.
2. Name the app, for example Chatbot.
3. Choose a warehouse (Small is enough).
4. Choose the CC_QUICKSTART_CORTEX_SEARCH_DOCS database and the DATA schema.
5. Delete the default code and paste in the contents of chatbot.py.
6. Open **Packages** and add **snowflake.core**. The combination tested in the Snowflake quickstart is snowflake.core 1.2.0, Python 3.11, snowflake-snowpark-python 1.30.0 and Streamlit 1.42.0.
7. Click **Run**.

The app does not need the snowflake-ml-python package, because it calls the model through SQL.

7.2 What users see

- **Welcome screen:** a banner and three suggested questions. It disappears as soon as a question is asked, whether typed or clicked.
- **Chat:** questions and answers appear in separate bubbles, with the input box at the bottom.
- **Sidebar settings:** only the controls people need.

Setting	What it does
Answer from my documents	On: answers use your PDFs. Off: the model answers from general knowledge only.
Product category	Limits the search to one category, or All.
Model	Chooses the LLM that writes the answer.
Remember this conversation	Lets the assistant use earlier messages for follow-up questions.
Passages per answer	How many chunks are retrieved as context (1 to 8, default 3).
Start over	Clears the conversation.

The app deliberately hides technical details such as the document list, the raw search JSON, session state and debug output, to keep the interface clean.

7.3 How a question is answered

1. **Take the history.** The last 7 messages (the slide window) are copied before the new question is added.
2. **Rewrite the question.** If memory is on and there is history, the model turns a follow-up such as "what about the mountain bike?" into a full, standalone search query.

3. **Retrieve.** Cortex Search returns the best chunks for that query, filtered by category if one is chosen.
4. **Build the prompt.** The chunks, the chat history and the question are combined. The model is told to answer only from the context, be concise, and say so when it does not know.
5. **Generate.** The model writes the answer, which is shown in the chat and saved in the session history.

7.4 Key code

Searching Cortex Search with an optional category filter

```
def search_chunks(query, category, num_chunks):
    svc = get_search_service()
    if category == 'All':
        response = svc.search(query, COLUMNS, limit=num_chunks)
    else:
        response = svc.search(
            query, COLUMNS,
            filter={'@eq': {'category': category}},
            limit=num_chunks,
        )
    return json.loads(response.json())['results']
```

Calling the model with AI_COMPLETE

```
def complete(model, prompt):
    row = session.sql(
        'select AI_COMPLETE(?, ?) as response',
        params=[model, prompt],
    ).collect()[0]
    return row['RESPONSE']
```

Values are passed as bound parameters, so apostrophes in questions and answers are safe and no quote-stripping is needed.

7.5 Why AI_COMPLETE

Snowflake is retiring the legacy SNOWFLAKE.CORTEX.COMPLETE function. New accounts are blocked from using it and see an error that tells them to migrate to AI Functions. The app therefore calls AI_COMPLETE, which accepts the same model and prompt arguments. The Python helper snowflake.cortex.Complete also calls the legacy function underneath, so the app does not use it.

7.6 Choosing models

The model list is the MODELS list at the top of chatbot.py. It currently contains llama3.1-70b, llama3.1-8b, snowflake-llama-3.3-70b, claude-4-sonnet and mistral-large2. Which models work depends on your region and account, so remove or replace any that return a "model not available" error.

8. Testing the Assistant

Questions to try (with the sample bike and ski documents)

- Is there any special lubricant to be used with the premium bike?
- Does the Mondracer infant bike need any special tool?

- Is there any temperature to be considered with the premium bicycle?
- What is the temperature to store the ski boots?
- Where have the ski boots been tested, and who tested them?

These facts exist only in the sample documents, so a correct answer shows the assistant is using your data.

Check that RAG is working. Turn **Answer from my documents** off and ask the same question. You should get a general, less specific answer. Turn it back on and the answer should use the details from your PDFs.

Check that memory is working. With **Remember this conversation** on, ask these two questions in a row:

1. Is there a special lubricant for the premium bike?
2. How often should I apply it?

If the second answer is about the lubricant, memory works. Click **Start over**, turn memory off and repeat: the second answer should now be vague, because "it" has no meaning on its own.

Note that asking "What was my first question?" may not work even when memory is on. The assistant is instructed to answer only from your documents and not to mention the chat history.

9. Keeping Documents Up to Date (Optional)

Cortex Search is a managed service, so keeping it current only means adding, changing or deleting rows in the chunks table. To automate this when files change in the stage:

1. Create two streams on the DOCS stage, one to catch inserted files and one for deleted files.
2. Create a stored procedure that deletes the chunks of removed files, parses and chunks new files, and classifies them.
3. Create a task on a schedule (for example every 5 minutes) that calls the procedure when the stream has data.

The Snowflake quickstart linked below contains the full SQL. Remember to suspend the task when you finish testing, so it does not keep using credits.

10. Troubleshooting

Problem	Likely cause and fix
Error: SNOWFLAKE.CORTEX.COMPLETE is not available on this account	The account blocks legacy functions. Use AI_COMPLETE, as chatbot.py does.
ModuleNotFoundError for snowflake.core	Add the snowflake.core package through the Packages menu and run again.
Model not available error	Remove or replace that model in the MODELS list. Check Cortex model availability for your region.

Problem	Likely cause and fix
Permission error when calling AI functions	Ask an administrator to grant the SNOWFLAKE.CORTEX_USER database role to your role.
Category dropdown is empty or shows only All	The category column in DOCS_CHUNKS_TABLE was not filled in. Re-run the classification and update steps.
Follow-up questions are not understood	Make sure Remember this conversation is on and that you did not click Start over.
Answers say the information is not available	The answer is not in the retrieved chunks. Try more passages, smaller chunks, or a different category filter.

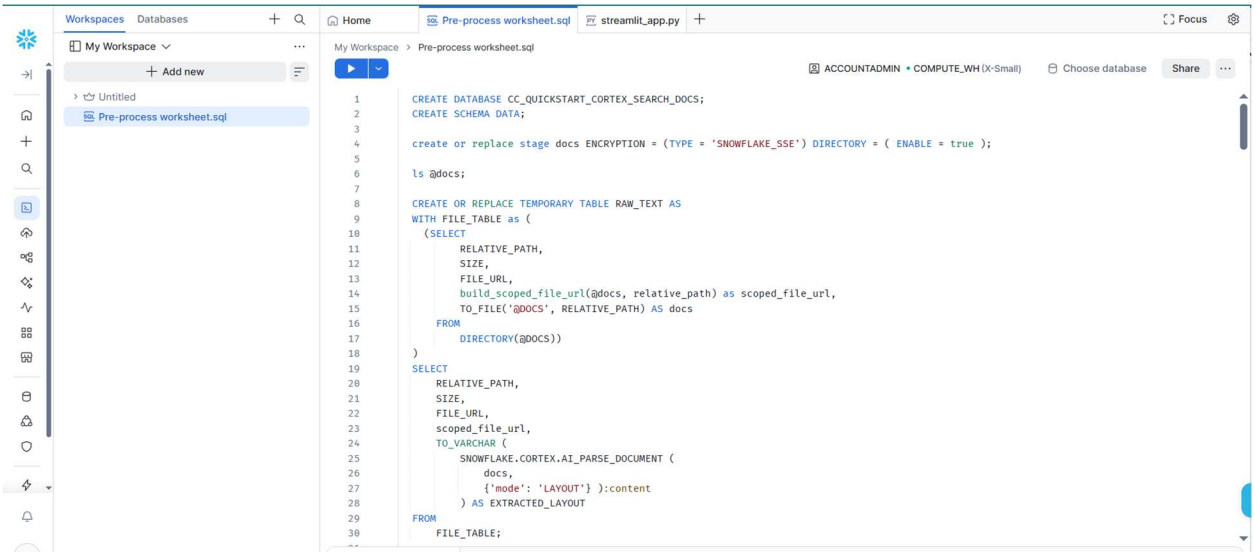
11. Cleanup

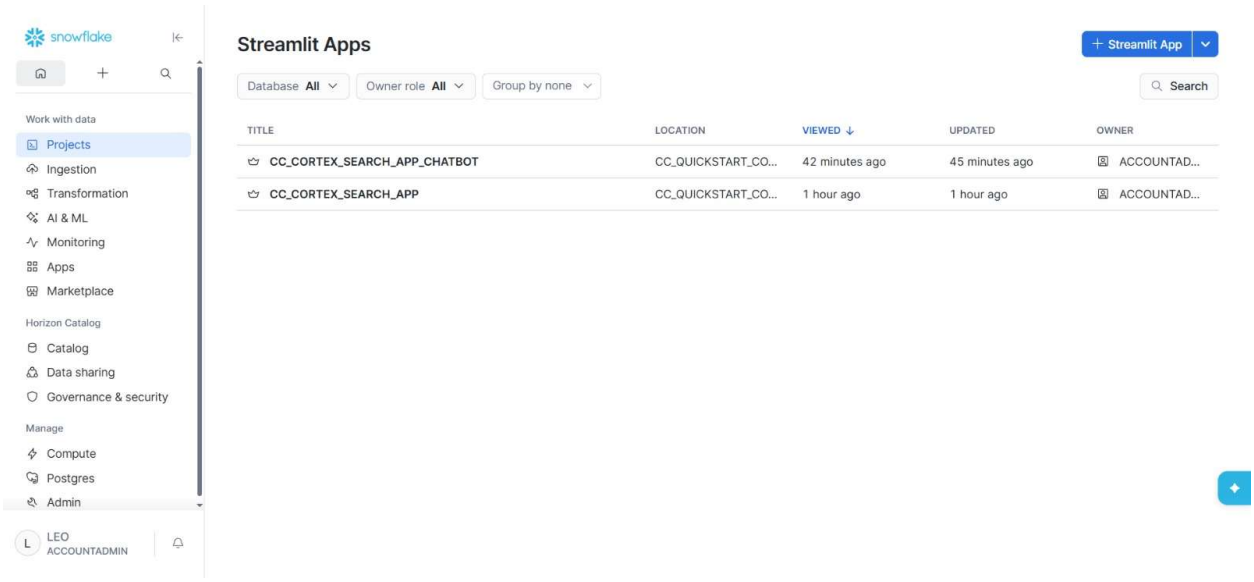
To avoid using credits when you are done, drop the database and suspend any tasks you created.

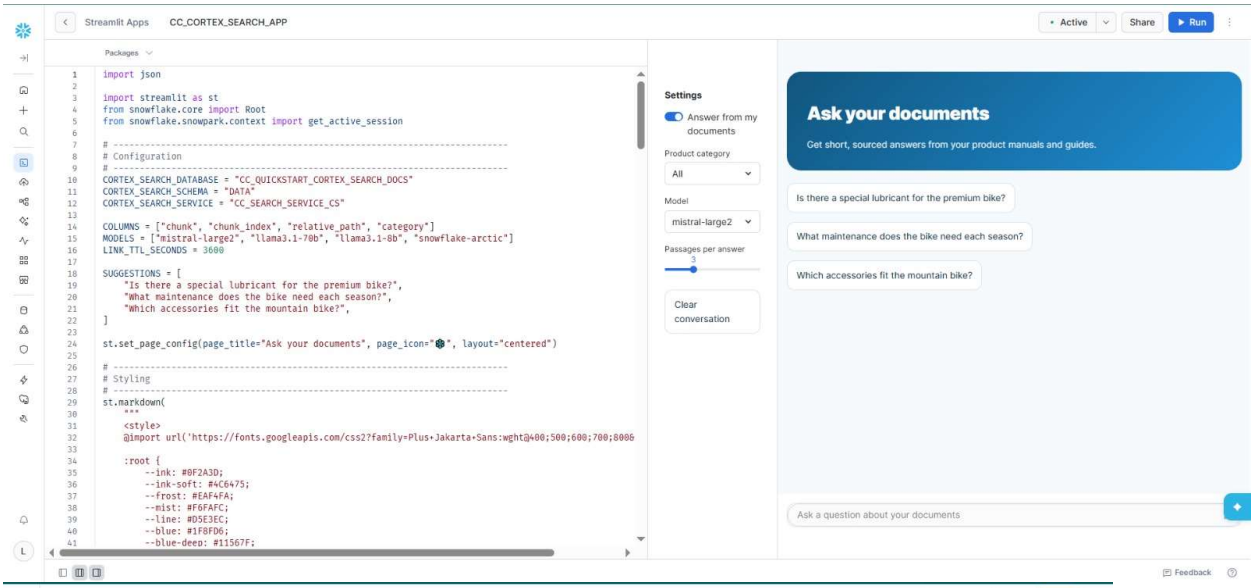
```
drop database CC_QUICKSTART_CORTEX_SEARCH_DOCS;
```

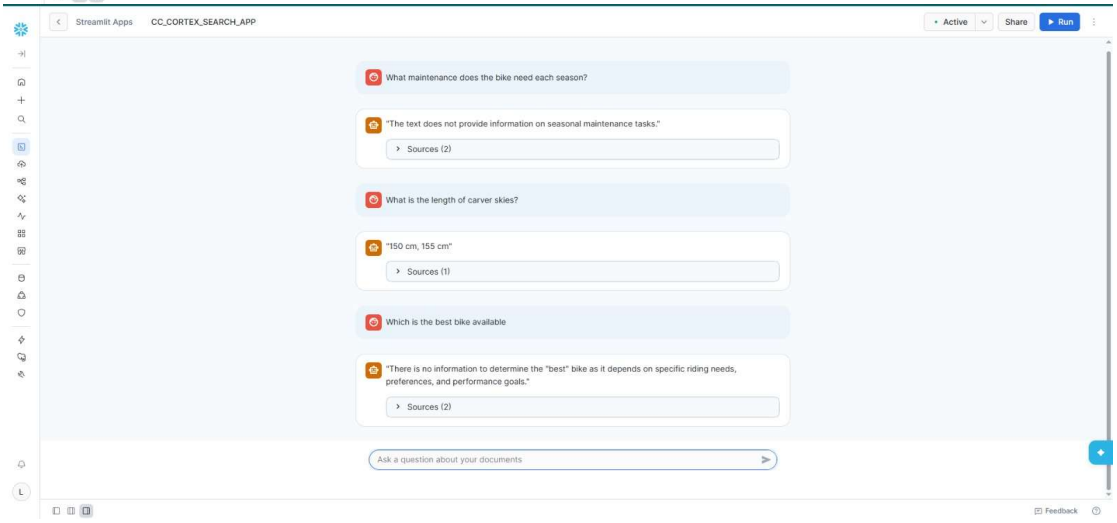
12. Snippets for reference:

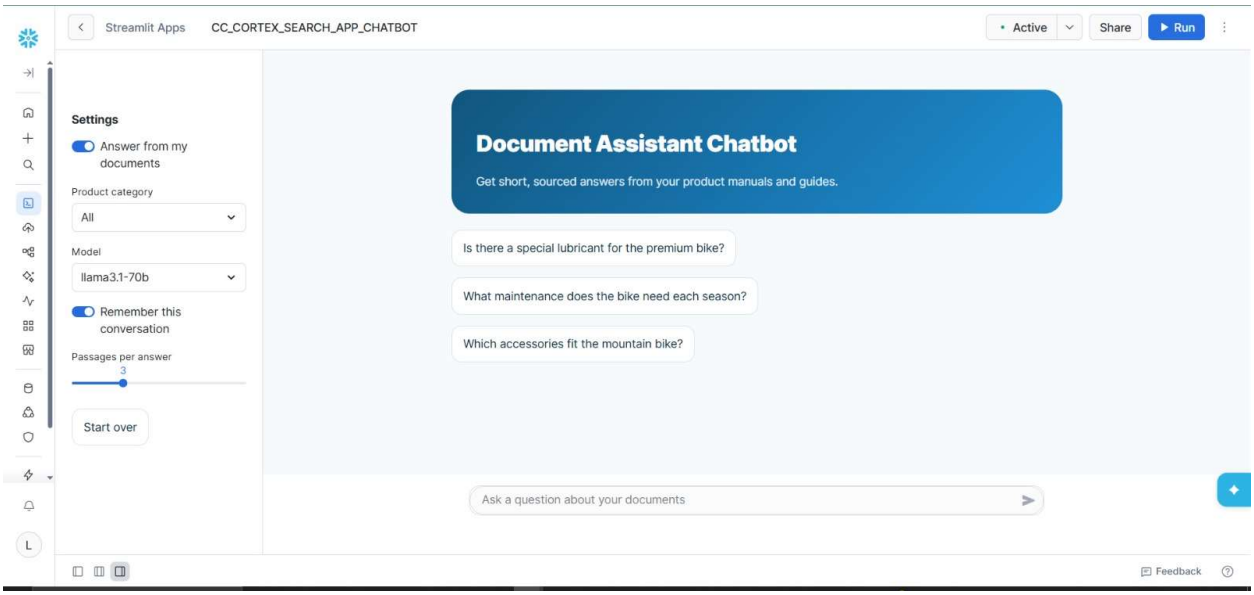
Set up:

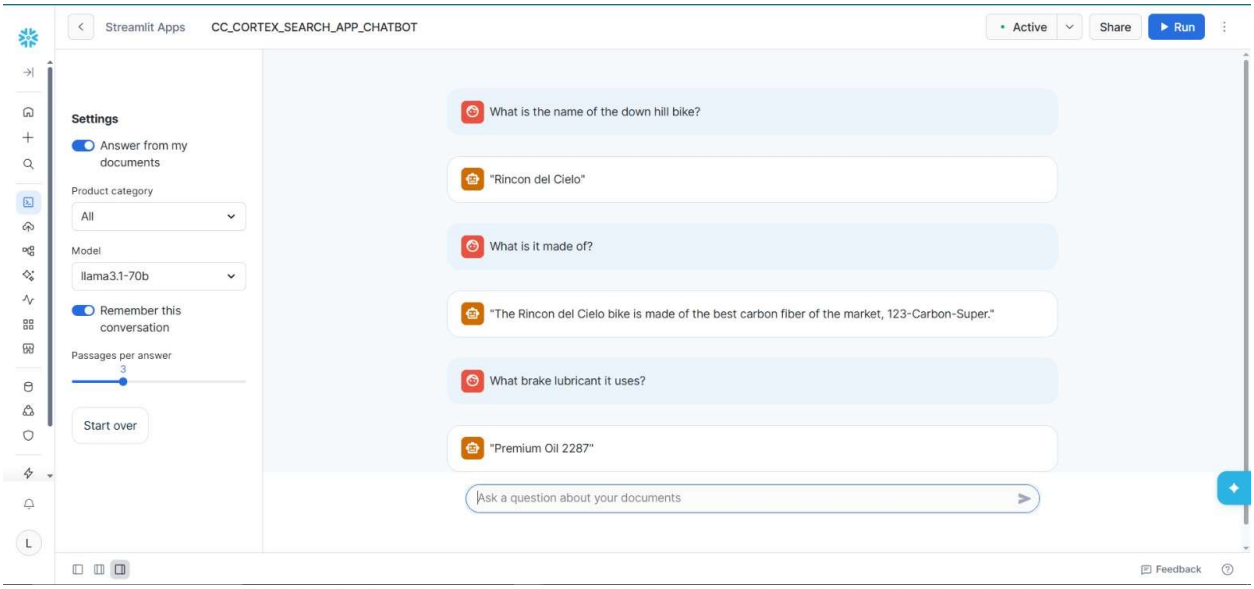


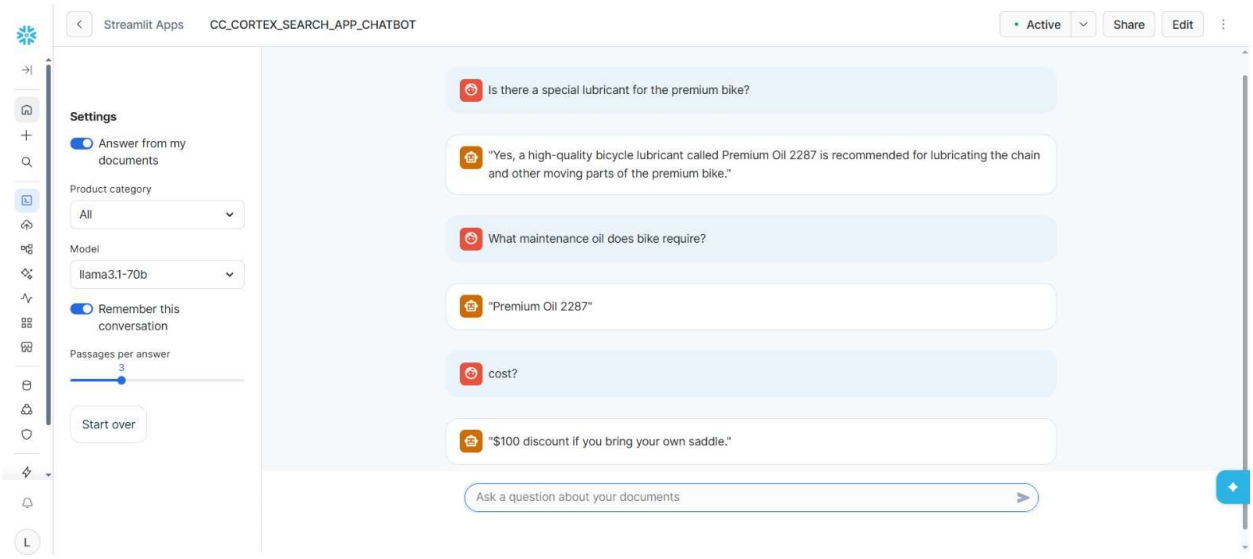

snowflake
Work with data
Projects
Ingestion
Transformation
AI & ML
Monitoring
Apps
Marketplace
Horizon Catalog
Catalog
Data sharing
Governance & security
Manage
Compute
Postgres
Admin
LEO
ACCOUNTADMIN
Streamlit Apps
+ Streamlit App
Database All
Owner role All
Group by none
Search
TITLE
LOCATION
VIEWED
UPDATED
OWNER
CC_CORTEX_SEARCH_APP_CHATBOT
CC_QUICKSTART_CO...
42 minutes ago
45 minutes ago
ACCOUNTAD...
CC_CORTEX_SEARCH_APP
CC_QUICKSTART_CO...
1 hour ago
1 hour ago
ACCOUNTAD...


Streamlit Apps
CC_CORTEX_SEARCH_APP
Active
Share
Run
Packages
import json
import streamlit as st
from snowflake.core import Root
from snowflake.snowpark.context import get_active_session
Configuration
CORTEX_SEARCH_DATABASE = "CC_QUICKSTART_CORTEX_SEARCH_DOCS"
CORTEX_SEARCH_SCHEMA = "DATA"
CORTEX_SEARCH_SERVICE = "CC_SEARCH_SERVICE_CS"
COLUMNS = ["chunk", "chunk_index", "relative_path", "category"]
MODELS = ["mistral-large2", "llama3.1-70b", "llama3.1-8b", "snowflake-arctic"]
LINK_TTL_SECONDS = 3600
SUGGESTIONS = [
"Is there a special lubricant for the premium bike?",
"What maintenance does the bike need each season?",
"Which accessories fit the mountain bike?",
]
st.set_page_config(page_title="Ask your documents", page_icon="❄", layout="centered")
Styling
st.markdown(
"""
<style>
@import url('https://fonts.googleapis.com/css2?family=Plus+Jakarta+Sans:wght@400;500;600;700;800
:root {
--ink: #0F2A3D;
--ink-soft: #4C6475;
--frost: #EAF4FA;
--mist: #F6FAFC;
--line: #D5E3EC;
--blue: #1F8FD6;
--blue-deep: #11567F;
Settings
Answer from my documents
Product category
All
Model
mistral-large2
Passages per answer
3
Clear conversation
Ask your documents
Get short, sourced answers from your product manuals and guides.
Is there a special lubricant for the premium bike?
What maintenance does the bike need each season?
Which accessories fit the mountain bike?
Ask a question about your documents
Feedback


Streamlit Apps
CC_CORTEX_SEARCH_APP
Active
Share
Run
What maintenance does the bike need each season?
"The text does not provide information on seasonal maintenance tasks."
Sources (2)
What is the length of carver skies?
"150 cm, 155 cm"
Sources (1)
Which is the best bike available
"There is no information to determine the "best" bike as it depends on specific riding needs, preferences, and performance goals."
Sources (2)
Ask a question about your documents
Feedback


Streamlit Apps
CC_CORTEX_SEARCH_APP_CHATBOT
Active
Share
Run
Settings
Answer from my documents
Product category
All
Model
llama3.1-70b
Remember this conversation
Passages per answer
3
Start over
Document Assistant Chatbot
Get short, sourced answers from your product manuals and guides.
Is there a special lubricant for the premium bike?
What maintenance does the bike need each season?
Which accessories fit the mountain bike?
Ask a question about your documents
Feedback


Streamlit Apps
CC_CORTEX_SEARCH_APP_CHATBOT
Active
Share
Run
Settings
Answer from my documents
Product category
All
Model
llama3.1-70b
Remember this conversation
Passages per answer
3
Start over
What is the name of the down hill bike?
"Rincon del Cielo"
What is it made of?
"The Rincon del Cielo bike is made of the best carbon fiber of the market, 123-Carbon-Super."
What brake lubricant it uses?
"Premium Oil 2287"
Ask a question about your documents
Feedback



13. Reference

- Snowflake quickstart: Build a Retrieval Augmented Generation (RAG) based LLM assistant using Streamlit and Snowflake Cortex Search (https://www.snowflake.com/en/developers/guides/ask-questions-to-your-own-documents-with-snowflake-cortex-search/)